

1 Интервальные запросы, LCA

1.1 Домашнее задание

1. Дано дерево из одной вершины. Требуется уметь отвечать online за $\mathcal{O}(\log n)$ на запросы:

- подвесить новую вершину u к вершине дерева v ,
- вернуть диаметр дерева.

Диаметр дерева — длина самого длинного простого пути в дереве.

2. Дан ориентированный граф, в котором исходящая степень каждой вершины равна единице. Запросы online: из вершины v сделать k шагов вперед и вернуть куда пришли.

(a) Предподсчет: $\mathcal{O}(n \log k_{\max})$, время на запрос: $\mathcal{O}(\log k)$.

(b) Предподсчет: $\mathcal{O}(n \log n)$, время на запрос: $\mathcal{O}(\log \min(k, n))$.

3. Дана скобочная последовательность из круглых скобок длины n . Запросы:

- является ли отрезок $[L, R]$ правильной скобочной последовательностью,
- изменить i -ю скобку.

$\mathcal{O}(n)$ на предподсчет, $\mathcal{O}(\log n)$ на запрос, online.

4. Попробуем модифицировать идею **SparseTable** так, чтобы она работала для произвольных ассоциативных функций: предложите способ выделить $\mathcal{O}(n \log n)$ отрезков в массиве размера n так, что любой отрезок $[L, R]$ можно было представить в виде объединения $\mathcal{O}(1)$ непересекающихся выделенных отрезков. Заметим, что дерево отрезков выделяет $\mathcal{O}(n)$ отрезков, и любой отрезок представляется как объединение $\mathcal{O}(\log n)$ из них.

Дополнительные задачи

5. Придумайте, как в задаче 4 домашнего задания обойтись $\mathcal{O}(n \log \log n)$ отрезков.

1.2 Решение

1. Мы будем хранить две вершины, между которыми сейчас диаметр в дереве и его значение.

Когда добавляется новая вершина, мы подвешиваем её к указанной, сохраняем глубину и предков для будущих запросов. Затем проверяем: а не стал ли диаметр больше, если пройти путь от новой вершины до одной из концов текущего диаметра. Если стал — обновляем диаметр.

Если мы добавляем новую вершину, то единственные кандидаты на новый диаметр — это пути от неё до текущих концов диаметра. Всё остальное уже проверено. Поэтому, проверяя только два таких пути, мы не упускаем новых вариантов. Расстояния между вершинами мы умеем находить быстро при помощи LCA и бинарных подъёмов.

2. (а) Пусть $up[v][i]$ — вершина, в которую попадаем, если из v сделать 2^i шагов.

$$\text{Предподсчёт: } \begin{cases} up[v][0] = f(v), \\ up[v][i] = up[up[v][i-1]][i-1] \end{cases}$$

Будем использовать бинарные подъёмы по степеням двойки от k .

- (б) Функциональный граф состоит из деревьев, прицепленных к циклам.

Если предварительно найти:

- Длину пути от каждой вершины до цикла (**depth**)
- Сам цикл и позицию вершины в нём

Если $k < \text{depth}[v]$, идём вверх по дереву как в прошлом пункте.

Иначе, делаем **depth**[v] шагов — попадаем в цикл.

Затем $k' = k - \text{depth}[v]$ шагов по циклу длины c - достаточно сделать $k' \bmod c$ шагов по циклу.

3. Используем дерево отрезков, где в каждой вершине храним два значения:

- **open** — количество лишних открывающих скобок на отрезке.
- **close** — количество лишних закрывающих скобок на отрезке.

Для одного символа:

- Если (, то **open** = 1, **close** = 0.
- Если), то **open** = 0, **close** = 1.

Научимся сливать отрезки.

Пусть левый сын: (**open**₁, **close**₁), правый сын: (**open**₂, **close**₂).

$$\text{Тогда: } \begin{cases} \text{match} = \min(\text{open}_1, \text{close}_2), \\ \text{open} = \text{open}_1 + \text{open}_2 - \text{match}, \\ \text{close} = \text{close}_1 + \text{close}_2 - \text{match} \end{cases}$$

Запрашиваем соответствующий отрезок в дереве.

Если результат (**open**, **close**) = (0, 0) — отрезок является правильной скобочной последовательностью.

При запросе на изменение, изменяем символ и пересчитываем значения в вершинах на пути от листа к корню дерева.

4. ...

5. ...